Institut Supérieur des Études Technologiques de Mahdia

Département : Technologies de l'Informatique

Développement des Systèmes Informatiques — DSI3.2

Rapport de Projet de Fin d'Études

Koda — Robot compagnon intelligent et émotionnel connecté, piloté par une application cross-platform

Système Embarqué · Intelligence Artificielle · Application Mobile Cross-Plateforme

Réalisé par :

Oussema KARIA

Salahedin JENDOUBI

Encadré par :

M.Wahid BANOUR — (Encadrant)

M.Hamza SASSI (Encadrant Entreprise)

Organisme d'accueil : —

Période : Février 2026 – Juin 2026

Abstract

This report presents KODA, an intelligent embedded life-assistant system designed to provide personalized support and companionship to users through a virtual assistant with its own unique nature and personality. KODA is capable of responding intelligently to any person, reasoning like a human, and recognizing individuals through its security and configuration modes.

The system is built around a multi-component architecture: a Raspberry Pi embedded system serving as the robot's physical core; an n8n workflow engine forming the brain of KODA, responsible for preparing and reasoning through responses; a Python service layer acting as a universal distributor, communicating with n8n, the database, the mobile application, and the Raspberry Pi while exposing all ecosystem endpoints; a cross-platform mobile application for configuration and life-monitoring; and a persistent database serving as KODA's memory. Developed following an agile methodology, the project ensures seamless integration between embedded hardware and scalable software components.

Keywords: Embedded System, Virtual Assistant, Raspberry Pi, n8n, Python, Cross-Platform, Artificial Intelligence, Life Assistant, KODA.

Résumé

Ce rapport présente KODA, un système embarqué intelligent conçu pour être un assistant de vie et un ami virtuel doté de sa propre personnalité et nature. KODA est capable de répondre de manière réfléchie à n'importe quelle personne, de raisonner comme un être humain et de reconnaître les individus grâce à ses modes de sécurité et de configuration.

L'architecture du système repose sur plusieurs composants complémentaires : un système embarqué Raspberry Pi constituant le cœur physique du robot ; un moteur de workflows n8n formant le cerveau de KODA, responsable de la préparation et de la réflexion des réponses ; une couche de services Python agissant comme distributeur universel, assurant la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi tout en exposant les endpoints de tout l'écosystème ; une application mobile cross-plateforme pour la configuration et le suivi de vie du robot ; et une base de données persistante servant de mémoire à KODA.

Mots-clés : Système Embarqué, Assistant Virtuel, Raspberry Pi, n8n, Python, Cross-Plateforme, Intelligence Artificielle, Assistant de Vie, KODA.

DÉDICACE

Je dédie ce travail à ma famille, pilier silencieux de ma force et source inépuisable de soutien. Merci pour vos sacrifices immenses et pour avoir cru en moi tout au long de mon parcours académique.

À mes parents, pour leur amour inconditionnel, leurs encouragements et leurs prières qui ont été la lumière guidant chacun de mes pas. Ce succès est le vôtre autant que le mien.

À mes amis et collègues, pour leur soutien moral et pour avoir partagé avec moi cette aventure inoubliable.

À mon binôme Salahedin, pour sa persévérance, sa créativité et sa complicité tout au long de ce projet.

À tous ceux qui ont contribué, même par un simple mot d'encouragement, à l'accomplissement de ce travail.

Oussema KARIA

DÉDICACE

À ma famille,
pour son soutien inconditionnel et ses sacrifices immenses
tout au long de mon parcours académique.

À mon binôme Oussema, pour son enthousiasme, sa rigueur
et sa précieuse collaboration dans la réalisation de ce projet.

À mes encadrants,
pour leur guidance, leur patience et leur expertise technique.

Salahedin JENDOUBI

Acknowledgements

Nous tenons à exprimer notre profonde et sincère gratitude à toutes les personnes qui ont contribué à la réalisation et au succès de ce projet.

Nous adressons nos remerciements les plus chaleureux à notre encadrant académique pour ses précieux conseils, sa disponibilité constante et son suivi rigoureux, garant de l'intégrité académique de notre travail.

Nous remercions également notre encadrant entreprise ainsi que toute l'équipe de l'organisme d'accueil pour leur accueil chaleureux, la mise à disposition des ressources nécessaires et les éclairages techniques partagés tout au long de notre stage.

Nous remercions par ailleurs la direction et le corps enseignant de l'ISET MAHDIA, département DSI, pour la formation de qualité qu'ils nous ont dispensée.

Enfin, nous remercions sincèrement nos amis et collègues pour leur soutien moral et pour avoir partagé avec nous cette aventure inoubliable.

CONTENTS

Introduction Générale **1**

I Cadre Général du Projet et Présentation de l'Organisme **2**

Introduction du chapitre 3

I.1 Contexte Général du Projet 3

I.2 Host Organization Overview (MicroEdition) 3

I.2.1 Company Profile 3

I.2.2 Services 4

I.2.3 Research & Development (R&D) 4

I.2.4 Organizational Structure 4

I.3 Problématique 5

I.4 Analyse des Solutions Existantes 5

I.4.1 Étude de l'existant 6

I.4.2 Synthèse et Critique : Le "Manque d'Humanité" 6

I.4.3 La Différenciation de Koda 7

I.5 Solution Proposée 7

I.6 Méthodologie du Projet 8

I.6.1 Approche Agile et Cycle en V 8

Conclusion 9

II Requirements Analysis and UML Modeling **10**

Introduction du chapitre 11

II.1 Requirements Analysis 11

II.1.1 Smart Greenhouse 11

II.1.2 Mobile Application (Farmer) 11

II.1.3 Web Application (Company/Admin) 11

II.1.4 AI Module (Disease Detection) 11

II.2 Functional and Non-Functional Requirements 12

II.3 Modeling Language 12

II.3.1 Why UML? 12
II.4 UML Design 12
II.4.1 Use Case Diagrams 13
II.4.2 Sequence Diagrams 13
II.4.3 Class Diagrams 14
Conclusion 14

III Analyseur et Réfléchissement 15
Introduction du chapitre 16
Définitions Fondamentales 16
Fondements Théoriques 17
Webhook 1 : Analyse et Réponse 18
Webhook 2 : Filtre et Synthèse Journalière 21
Webhook 3 : Spontanéité et Interaction Proactive 23
Architecture Cognitive Globale de KODA 24
Conclusion 24

IV Smart Greenhouse & IoT Infrastructure 25
Chapter Introduction 26
IV.1 Overall IoT System Architecture 26
IV.2 Hardware Components 26
IV.2.1 Raspberry Pi Integration 26
IV.2.2 Sensors and Actuators 26
IV.3 Data Acquisition and Communication 26
IV.3.1 MQTT Protocol 27
IV.3.2 Broker and Data Exchange 27
IV.4 Physical Implementation and Wiring 27
Conclusion 27

V Artificial Intelligence Module (Research & Development) 28
Chapter Introduction 29
V.1 AI Approach Study 29
V.2 Data Preparation (Dataset) 29
V.2.1 Data Collection and Sources 29
V.2.2 Labeling and Preprocessing 29

V.3 Model Development 29
V.3.1 Training and Testing 30
V.3.2 Pipeline Edge Impulse 30
V.4 Edge Deployment and OTA Updates 30
Conclusion 30

VI Mobile Application (Farmer) & Web Dashboard (Admin) 31
Chapter Introduction 32
VI.1 Mobile Application Interface 32
VI.1.1 Real-Time Monitoring 32
VI.1.2 Notifications and Alerts 32
VI.2 Web Administration Dashboard 32
VI.2.1 User and Device Management 33
VI.2.2 Data Visualization (Grafana) 33
VI.3 Video Surveillance System 33
VI.3.1 Architecture WebRTC / Coturn 33
VI.3.2 Live Greenhouse Monitoring 33
Conclusion 33

VIIDeployment and Cloud Integration 34
Chapter Introduction 35
VII.1 Cloud Deployment Architecture 35
VII.2 Backend & Database 35
VII.2.1 Architecture Spring Boot 35
VII.2.2 Time-Series Data Management (TimescaleDB/PostgreSQL) 35
VII.3 Security and Authentication 36
VII.4 Full System Validation and Testing 36
Conclusion 36

General Conclusion and Future Perspectives 38

Bibliography 40

Webography 42

LIST OF FIGURES

I.1 Architecture Globale de la Solution Proposée 8

II.1 Diagramme de cas d'utilisation — Systèmme de gestion de serre connectée 13

II.2 Global Use Case interaction paths across distinct administrative roles. 13

LIST OF TABLES

II.1 Explicit Quality of Service (QoS) and non-functional requirements. 12

III.1 Correspondance entre mécanismes cognitifs humains et implémentations de KODA 24

INTRODUCTION GÉNÉRALE

Dans un monde en constante évolution technologique, la demande en solutions intelligentes capables d'améliorer la qualité de vie des individus n'a jamais été aussi forte. Les assistants virtuels et les systèmes embarqués intelligents représentent aujourd'hui une réponse concrète à ce besoin, en combinant les avancées de l'intelligence artificielle, des systèmes embarqués et du développement logiciel moderne.

Ce projet de fin d'études, réalisé dans le cadre de la formation en Développement des Systèmes Informatiques (DSI3.2) à l'ISET Mahdia, porte sur la conception et le développement de **KODA** : un assistant de vie et ami virtuel embarqué. KODA est un système robotique doté de sa propre personnalité et nature, capable de répondre intelligemment à toute personne, de raisonner à la manière d'un être humain, de reconnaître les individus à travers ses modes de sécurité, et d'être configuré et suivi via une application mobile cross-plateforme dédiée.

La problématique centrale de ce projet est : **Comment concevoir un système embarqué intelligent capable d'assurer un rôle d'assistant de vie personnalisé, intégrant reconnaissance des personnes, réflexion autonome et pilotage à distance via une application mobile ?**

Ce rapport est organisé comme suit :

- **Chapitre I** présente le cadre général du projet, l'organisme d'accueil, la problématique, l'analyse de l'existant et la solution proposée.
- **Chapitre II** définit les besoins fonctionnels et non fonctionnels ainsi que la modélisation UML du système.
- **Chapitre III** décrit l'architecture globale du système KODA et l'interaction entre ses différentes couches.
- **Chapitre IV** détaille le système embarqué Raspberry Pi, son rôle et son implémentation.
- **Chapitre V** présente le moteur de traitement n8n et la couche Python, cœur logique de KODA.
- **Chapitre VI** couvre l'application mobile cross-plateforme de configuration et de suivi.
- **Chapitre VII** traite du déploiement, de la base de données et des tests de validation du système complet.

CHAPTER I

CADRE GÉNÉRAL DU PROJET ET PRÉSENTATION DE L'ORGANISME

Introduction du chapitre

Ce chapitre esquisse le cadre fondamental de notre étude en définissant l'environnement contextuel et organisationnel du projet. Nous commençons par introduire l'organisme d'accueil, en mettant en évidence son expertise technologique et sa structure interne dédiée à l'innovation.

Ensuite, nous exposons la problématique centrale, justifiant le besoin critique d'un système intelligent. Une analyse rigoureuse des solutions existantes nous permet d'identifier les limites actuelles et de mieux positionner notre système proposé. Enfin, nous détaillons la méthodologie de projet adoptée, assurant une approche structurée et efficace tout au long du cycle de développement.

I.1 Contexte Général du Projet

L'évolution technologique actuelle transforme notre interaction avec l'environnement. La robotique sociale et les assistants personnels virtuels s'imposent comme des éléments clés pour améliorer la qualité de vie, offrant des solutions innovantes pour l'assistance quotidienne.

Le projet KODA s'inscrit dans cette dynamique en proposant un assistant de vie virtuel, conçu sous la forme d'un système embarqué intelligent. Contrairement aux solutions traditionnelles qui se limitent à l'exécution de commandes, KODA vise à offrir une véritable présence interactive, dotée d'une personnalité propre et d'une capacité de raisonnement.

La complexité de ce système réside dans l'intégration harmonieuse de multiples composants hétérogènes. Il nécessite la combinaison d'un environnement matériel embarqué (Raspberry Pi), d'un moteur de workflow intelligent (n8n) agissant comme cerveau, d'une couche logicielle orchestrant les communications (Python), et d'interfaces de contrôle intuitives (application mobile croisée). L'objectif est de créer un écosystème cohérent capable de reconnaître les utilisateurs, de traiter les requêtes de manière naturelle, et de fournir un accompagnement personnalisé.

I.2 Host Organization Overview (MicroEdition)

I.2.1 Company Profile

MicroEdition is a pioneering enterprise specializing in Information and Communication Technologies (ICT). Established in 2025, the company is strategically headquartered within the Mahdia Business Incubator. This environment provides a robust ecosystem for innovation, offering the necessary infrastructure to support high-potential technology startups. MicroEdition is dedicated to the architecture and deployment of intelligent, scalable, and secure digital solutions designed to bridge the gap between academic research and industrial application.

I.2.2 Services

The company's portfolio is built upon three fundamental technological pillars:

- **Software Development:** Crafting high-performance web and mobile applications tailored to specific business logic.
- **Internet of Things (IoT):** Designing end-to-end connected ecosystems for industrial and agricultural automation.
- **Cybersecurity:** Implementing rigorous security protocols to protect digital assets and ensure data integrity within private and public sectors.

I.2.3 Research & Development (R&D)

A distinguishing feature of MicroEdition is its specialized Research and Development division. This department focuses heavily on Edge AI and Embedded Intelligence. By investigating methodologies to execute complex neural networks locally on microcontrollers, the R&D team seeks to eliminate the dependency on constant cloud connectivity. This strategic focus is central to this project, as it enables the creation of autonomous, resilient systems capable of operating in remote agricultural environments where internet access may be intermittent.

I.2.4 Organizational Structure

MicroEdition operates through an Agile-driven framework that emphasizes rapid iteration and interdisciplinary collaboration. The company's internal structure is composed of specialized units, including:

- **Software Architects:** Designing scalable system backends.
- **Embedded Systems Engineers:** Bridging the gap between hardware and software.
- **AI and Data Scientists:** Developing and optimizing machine learning models for edge deployment.

This synergy allows MicroEdition to maintain a fast-paced innovation cycle, ensuring that every project—such as the Intelligent Hydroponic Fodder System—is built on a foundation of technical excellence and modern architectural standards.

I.3 Problématique

L'évolution constante de notre société moderne, marquée par un rythme de vie accéléré et une digitalisation croissante, a paradoxalement accentué certains besoins fondamentaux de l'être humain, notamment celui de l'accompagnement et de l'interaction sociale.

Le problème central auquel nous tentons de répondre s'articule autour de trois axes principaux :

A. Le besoin de présence et d'interaction naturelle

Malgré la prolifération des appareils connectés, la plupart des outils actuels (smartphones, enceintes connectées classiques) restent des outils purement fonctionnels. Ils exécutent des ordres mais n'offrent pas de réelle interaction "humaine". Il existe un manque flagrant de systèmes capables d'afficher une personnalité propre et de simuler une forme de réflexion empathique capable de briser la solitude ou d'assister l'utilisateur au quotidien.

B. La complexité de la gestion de vie autonome

Dans un environnement domestique, la gestion des tâches, de la sécurité et du suivi technique des objets connectés devient de plus en plus complexe pour l'utilisateur moyen. Il n'existe pas de solution centralisée qui combine à la fois un assistant de vie intelligent (capable de raisonner) et un système de sécurité actif (capable de reconnaître les personnes et de protéger l'habitat).

C. Les limites de l'intelligence statique

La majorité des assistants actuels dépendent de réponses préprogrammées et rigides. Le défi est de passer d'une machine qui "répond" à une machine qui "réfléchit" et "apprend". Le problème est donc de concevoir une architecture capable de :

- Identifier l'interlocuteur de manière sécurisée.
- Traiter des demandes complexes de manière fluide et humaine.
- Offrir une interface de contrôle simple (application mobile) pour superviser cette "vie artificielle".

En résumé, la question fondamentale de notre projet est la suivante : **Comment concevoir un compagnon robotique intelligent, doté d'une mémoire et d'une personnalité, capable d'offrir un accompagnement humain authentique tout en assurant une gestion technique et sécurisée de l'environnement de l'utilisateur ?**

I.4 Analyse des Solutions Existantes

I.4.1 Étude de l'existant

Dans cette section, nous examinons les assistants domestiques et les robots compagnons les plus populaires sur le marché afin d'identifier leurs forces et leurs lacunes.

- **A. Les Assistants Vocaux Statiques (Amazon Alexa, Google Home)**

  Ces dispositifs sont les plus répandus. Ils excellent dans l'exécution de tâches simples (domotique, rappels, recherches internet).

  Limites : Ils n'ont pas de présence physique mobile. Surtout, ils restent des outils purement réactifs : ils attendent un "mot déclencheur" et répondent de manière standardisée. Il n'y a aucune notion de "personnalité" évolutive ou de sentiment d'appartenance.

- **B. Les Robots Domestiques (ex: Amazon Astro, Vector/Cozmo)**

  Certains robots essaient d'occuper l'espace physique. Amazon Astro, par exemple, surveille la maison et suit l'utilisateur.

  Limites : Bien que mobiles, ces produits restent perçus comme des gadgets technologiques. Leur interaction est limitée par des scripts pré-enregistrés. Ils ne "réfléchissent" pas de manière humaine et leur capacité à reconnaître et s'adapter à l'humeur ou à l'identité spécifique de chaque membre de la famille de façon approfondie reste superficielle.

- **C. Les Pepper & Nao (Robotique de service)**

  Ce sont des robots haut de gamme capables de simuler des émotions et de reconnaître des visages.

  Limites : Ces solutions sont extrêmement coûteuses (plusieurs milliers d'euros) et sont destinées aux entreprises plutôt qu'au grand public. Leur configuration est complexe et ils ne sont pas conçus comme des "amis" personnels intégrés à un écosystème mobile fluide pour l'utilisateur lambda.

I.4.2 Synthèse et Critique : Le "Manque d'Humanité"

Le point commun de tous ces concurrents est leur nature générique. Ce sont des machines produites en série qui traitent chaque utilisateur de la même manière.

- **Absence de personnalité :** La réponse d'une Alexa est la même pour tout le monde. Elle n'a pas de "caractère" propre qui pourrait la faire passer pour un compagnon de vie.
- **Froideur algorithmique :** Les interactions sont basées sur des arbres de décision rigides. L'utilisateur sent qu'il parle à une base de données, pas à une entité qui "réfléchit".

- **Dépendance au Cloud propriétaire :** L'utilisateur a peu de contrôle sur la "mémoire" ou la logique interne de l'assistant.

I.4.3 La Différenciation de Koda

C'est ici que notre projet se distingue. Contrairement aux machines industrielles, Koda est conçu pour être :

- **Unique :** Grâce à l'intégration de n8n (le cœur de réflexion), Koda peut avoir une logique de réponse personnalisée et évolutive.
- **Multimodal :** Il combine la vision (Raspberry Pi), la réflexion (IA via n8n) et la mobilité.
- **Accessible :** Une application mobile dédiée permet de configurer sa "personnalité", rendant l'utilisateur maître de son compagnon.

I.5 Solution Proposée

Pour surmonter ces limitations, nous proposons KODA, une architecture résiliente et multi-couche conçue pour l'autonomie et l'intelligence. La solution comprend :

- **Contrôleur Embarqué (Raspberry Pi) :** Le cœur physique du système, gérant les composants matériels et assurant le fonctionnement du robot.
- **Moteur de Réflexion (n8n) :** Le cerveau de KODA, responsable de la préparation des réponses et du traitement intelligent des requêtes complexes.
- **Couche de Distribution (Python) :** Le système nerveux responsable de la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi, offrant les points d'accès (endpoints) de tout l'écosystème.
- **Expérience Utilisateur Mobile :** Une application cross-plateforme permettant à l'utilisateur de configurer, contrôler et suivre la vie de son compagnon robotique en temps réel.
- **Mémoire Persistante (Base de données) :** Un système de stockage permettant à KODA d'enregistrer des informations contextuelles et d'apprendre de ses interactions.

En intégrant ces composants, la solution proposée garantit une résilience opérationnelle, des interactions humaines authentiques et une expérience de gestion unifiée.

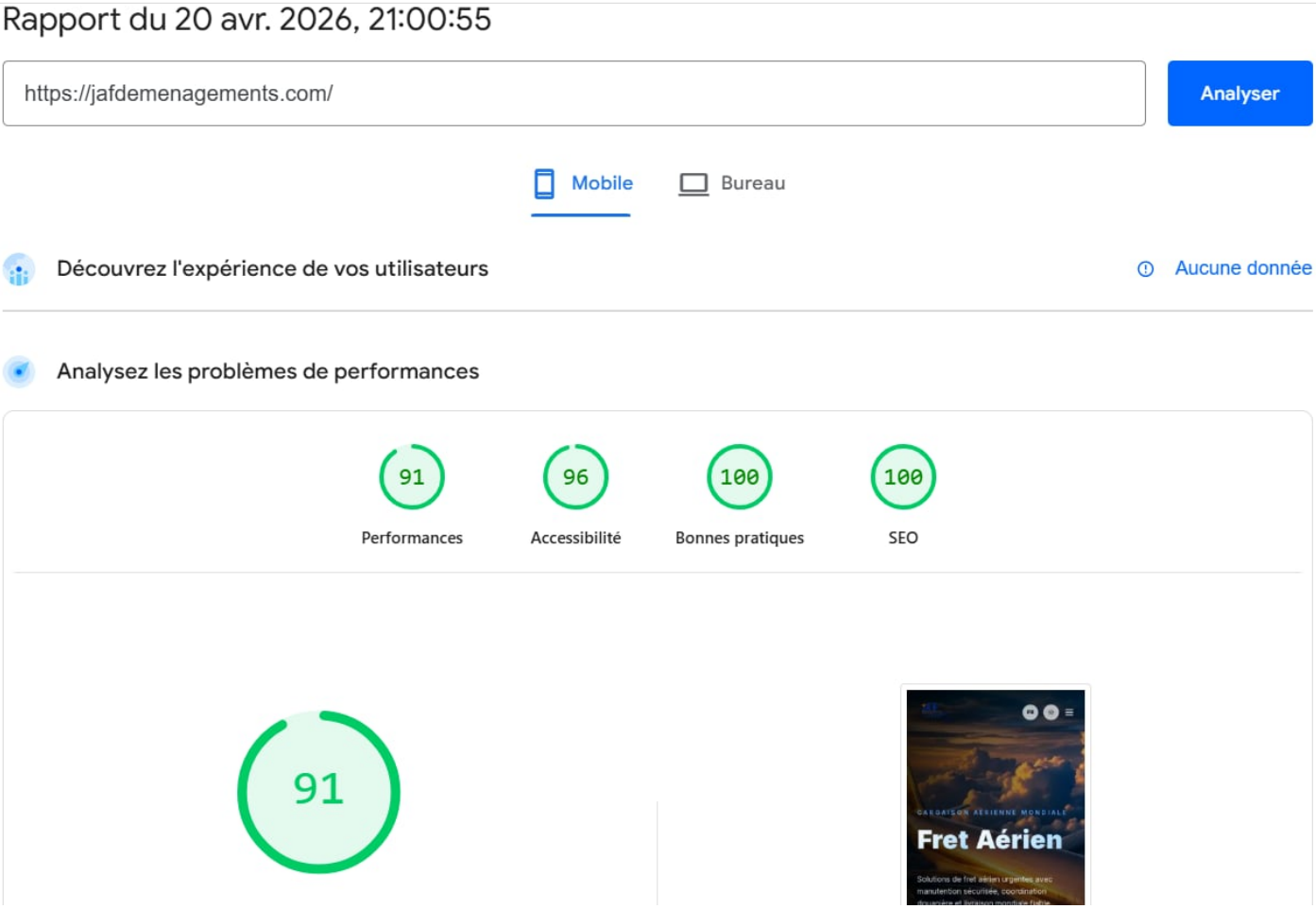

Figure I.1 – Architecture Globale de la Solution Proposée

I.6 Méthodologie du Projet

Pour garantir un haut degré de fiabilité et une intégration harmonieuse des différentes couches logicielles et matérielles, ce projet suit un cadre d'ingénierie rigoureux.

I.6.1 Approche Agile et Cycle en V

Bien que le projet intègre des concepts de prototypage rapide caractéristiques des méthodes Agiles, l'architecture globale repose sur un Cycle en V. Ce modèle représente un chemin d'exécution séquentiel où chaque phase de développement (côté gauche) est associée à une phase de test correspondante (côté droit).

Nous avons adopté cette méthodologie car elle garantit que les protocoles de test pour notre matériel embarqué et nos workflows d'intelligence artificielle soient définis avant l'assemblage physique, conduisant à un cycle d'intégration plus efficace. La figure **??** illustre les étapes fondamentales de ce cadre dans un contexte d'ingénierie général.

1) Le flux de processus : Déconstruction et Réassemblage

La méthodologie forme une forme en "V" symétrique, divisée en trois segments principaux :

- **Le segment de vérification (Étapes 1–4) :** Se concentre sur la traduction des besoins des utilisateurs en un plan architectural définitif.
- **Le sommet (Étape 5) :** L'implémentation est le tournant critique où le codage, l'assemblage matériel et la configuration des workflows ont lieu.
- **Le segment de validation (Étapes 6–8) :** Réassemble et vérifie le système à travers des tests unitaires et des tests d'intégration globaux.

2) Artefacts Clés du Projet

La traçabilité est maintenue en générant des artefacts spécifiques à chaque étape :

- **Spécifications des Besoins :** Besoins utilisateurs et techniques documentés.
- **Conception de l'Architecture Système :** Plans d'intégration matériel/logiciel.
- **Conception Détaillée des Composants :** Logique de code, endpoints API et schéma de base de données.

Conclusion

Ce chapitre a établi le contexte du projet KODA. En identifiant les lacunes critiques des assistants virtuels actuels — notamment le manque de personnalité, de réflexion autonome et d'intégration embarquée — nous avons défini notre solution innovante. De plus, en adoptant une méthodologie de développement structurée, nous avons établi une feuille de route qui garantit la fiabilité tout au long des défis techniques à venir.

Le chapitre suivant, "Étude des Besoins et Modélisation UML", initiera la première étape de ce cycle de conception en transformant les besoins de l'utilisateur en spécifications définitives et en modèles architecturaux.

CHAPTER II

REQUIREMENTS ANALYSIS AND UML MODELING

Introduction du chapitre

Designing an advanced agricultural solution requires a rigorous analysis of user needs and a formal software blueprint. This chapter comprehensively defines the functional and non-functional requirements for each system actor, then formalizes the conceptual framework through UML modeling to map user interactions, behavioral sequences, and structural class relationships.

II.1 Requirements Analysis

The system caters explicitly to four primary actors, each with distinct operational demands:

II.1.1 Smart Greenhouse

The smart greenhouse represents the physical automated environment. It requires autonomous edge-control to prevent catastrophic pump failures and demands millisecond-level IoT interaction arrays. The system must continuously monitor and regulate temperature, humidity, water levels, and lighting cycles without human intervention, while maintaining full operational capability during internet outages.

II.1.2 Mobile Application (Farmer)

The farmer requires a simple, intuitive mobile interface capable of sending manual override commands and receiving immediate AI-driven push notifications. The application must display real-time telemetry data, live video streaming from the greenhouse camera, and actionable disease alerts in an accessible format.

II.1.3 Web Application (Company/Admin)

MicroEdition's administrative team requires a secure, multi-tenant web interface providing a global overview of all operational greenhouses. This includes full user account management, device configuration, historical data reporting, and system-wide alert supervision.

II.1.4 AI Module (Disease Detection)

A critical requirement is the integration of an embedded Artificial Intelligence classifier. The model must autonomously capture live camera snapshots of the fodder leaves, perform convolutional image analysis to detect mold or decay, and instantly trigger localized emergency ventilation overrides while sending severe alerts to the farmer's mobile device.

II.2 Functional and Non-Functional Requirements

Functional Requirements:

- Secure multi-role authentication (JWT).
- Real-time telemetry monitoring displaying precise humidity, temperature, and water levels.
- Remote control overriding fans, lights, and irrigation pumps.
- AI-powered plant disease detection with instant alert notifications.
- Complete CRUD management for user accounts and greenhouse assignments.
- Live video streaming from the greenhouse camera.
- Historical data visualization and export capabilities.

Non-Functional Requirements:

Table II.1 – Explicit Quality of Service (QoS) and non-functional requirements.

Category	Requirement Parameter	Target Threshold
Performance	API HTTP Response Times	$< 500\,\text{ms}$
Performance	Streaming latency (WebRTC)	$< 2\,\text{s}$ lag
Security	Data Transport	HTTPS / WSS / MQTTS
Security	Auth Flow	JWT & standard OAuth2
Availability	Hardware Uptime	$\geq 99.5\%$
Scalability	Data Ingestion	Thousands of metrics/min

II.3 Modeling Language

II.3.1 Why UML?

The Unified Modeling Language (UML) is the industry-standard notation for object-oriented software engineering. Employing UML eliminates architectural ambiguity, establishes a clear visual understanding between engineers and stakeholders, and accelerates implementation by defining interaction sequences prior to development. Its standardized notation ensures clarity for all technical and non-technical stakeholders involved in the project.

II.4 UML Design

II.4.1 Use Case Diagrams

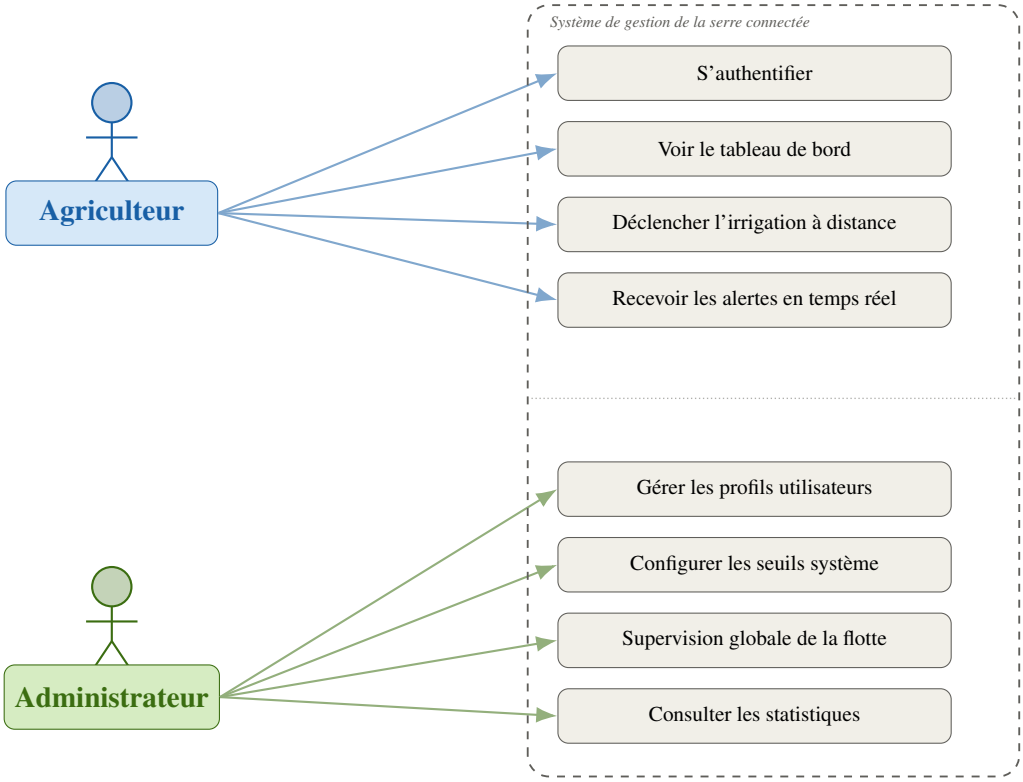

Figure II.1 – Diagramme de cas d'utilisation — Systèmme de gestion de serre connectée

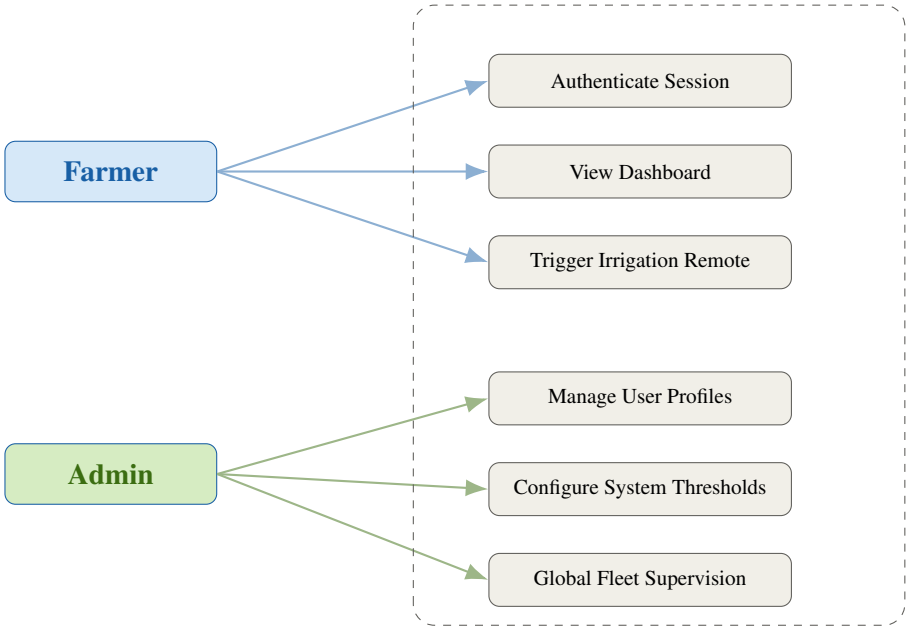

Figure II.2 – Global Use Case interaction paths across distinct administrative roles.

II.4.2 Sequence Diagrams

Sequence diagrams capture the temporal logic of system interactions. For example, a remote pump actuation begins when the farmer initiates a request via the Flutter UI. The request is

routed through the API Gateway to the Spring Boot core, which issues a secure MQTT command. The Raspberry Pi receives this payload, toggles the corresponding GPIO pin, and returns an acknowledgment confirming the physical actuation back to the mobile interface.

II.4.3 Class Diagrams

The class diagram defines the structural relationships underpinning the platform. The core entities include **User** (containing credentials and role designations), **Greenhouse** (holding physical location and status attributes), and **Sensor Array** mapping multiple **Telemetry** outputs linked to timestamps representing historical data boundaries. These relationships enforce strict data integrity across the entire system.

Conclusion

This chapter defined the precise needs of each system actor and formalized the system's conceptual architecture through comprehensive UML modeling. Having structured the relational database mapping and visualized user interaction sequences, the subsequent phases can proceed to the implementation of core infrastructure across both embedded controllers and server ecosystems.

CHAPTER III

ANALYSEUR ET RÉFLÉCHISSEMENT

Introduction du chapitre

Ce chapitre présente le moteur central de KODA : son système d'analyse, de réflexion et de génération de réponses. L'objectif principal est de permettre au robot de répondre de manière pertinente, contextualisée et naturelle aux questions posées par les utilisateurs. KODA étant conçu comme un accompagnateur virtuel à dimension humaine, il doit comprendre non seulement le contenu des questions, mais aussi le contexte et les besoins implicites de chaque utilisateur.

Pour atteindre cet objectif, nous avons conçu une architecture logicielle organisée autour de trois flux de travail automatisés (appelés **webhooks**), chacun remplissant une fonction précise et complémentaire. Avant de détailler leur fonctionnement, nous commençons par définir les deux technologies fondamentales sur lesquelles repose l'ensemble du système : **n8n** et les **webhooks**.

Définitions Fondamentales

Qu'est-ce que n8n ?

n8n (prononcé "n-eight-n", abréviation de *nodemation*) est une **plateforme d'automatisation de flux de travail open-source**, conçue pour connecter des applications, des APIs et des services entre eux sans nécessiter de développement complexe.

Techniquement, n8n fonctionne selon une architecture orientée **nœuds** (nodes) : chaque nœud représente une opération atomique — un appel à une API, une transformation de données, une condition logique, ou l'invocation d'un modèle d'intelligence artificielle. Ces nœuds sont reliés entre eux pour former des **flux de traitement** (workflows), dans lesquels les données circulent sous forme d'objets JSON structurés d'un nœud à l'autre.

Voici les caractéristiques essentielles qui ont motivé notre choix de n8n pour le projet KODA :

- **Architecture DAG (Graphe Orienté Acyclique) :** Les nœuds sont organisés en graphe sans cycle, garantissant un traitement ordonné et prévisible des données.
- **Auto-hébergement :** Contrairement à des outils comme Zapier ou Make, n8n peut être installé sur un serveur privé. Cela assure un contrôle total sur les données des utilisateurs — un critère indispensable pour un projet traitant des informations personnelles.
- **Intégration native avec les modèles IA :** n8n propose des connecteurs prêts à l'emploi pour les principaux modèles de langage (OpenAI, Anthropic Claude, Mistral), permettant d'insérer des nœuds d'intelligence artificielle directement dans le flux de traitement.
- **Interface visuelle :** Le concepteur de workflows est entièrement graphique, permettant de visualiser, tester et déboguer chaque étape du traitement en temps réel.

- **Déclencheurs multiples :** n8n supporte des dizaines de modes de déclenchement — webhooks HTTP, planification cron, événements base de données, etc.

Référence : Documentation officielle n8n — `https://docs.n8n.io`

Qu'est-ce qu'un Webhook ?

Un **webhook** est un mécanisme de communication entre deux systèmes informatiques basé sur le modèle **événement** → **notification** → **action**. Contrairement à une requête classique où c'est le client qui interroge régulièrement un serveur pour savoir si quelque chose a changé (modèle *polling*), un webhook fonctionne à l'inverse : c'est le serveur émetteur qui notifie automatiquement le serveur récepteur dès qu'un événement se produit.

Concrètement, un webhook est une **URL HTTP unique** exposée par le système récepteur (ici, n8n). Lorsqu'un événement survient dans le système émetteur (par exemple : l'utilisateur pose une question au robot), celui-ci envoie une requête HTTP POST vers cette URL, avec les données de l'événement dans le corps de la requête. Le serveur récepteur traite alors ces données et déclenche le pipeline de traitement associé.

Concept	Signification
URL du webhook	Adresse unique qui reçoit les notifications
Déclencheur	L'événement qui provoque l'envoi
Payload	Les données transmises (format JSON)
Pipeline	L'ensemble des traitements activés

Dans KODA, nous utilisons trois webhooks distincts, chacun dédié à une fonction cognitive précise du robot.

Référence : Mozilla Developer Network — `https://developer.mozilla.org/en-US/docs/Glossary/W`

Fondements Théoriques : Méthodes de Réflexion et de Traitement de l'Information

La Théorie du Double Traitement

Le cadre théorique central de notre conception est la **théorie du double traitement** (*Dual-Process Theory*), formalisée par le psychologue et prix Nobel d'économie Daniel Kahneman. Cette théorie distingue deux systèmes de pensée fondamentalement différents dans le cerveau humain :

- **Système 1 — Rapide et Automatique :** Ce système réagit de manière instinctive, sans effort conscient. Il s'appuie sur des associations mémorisées et des réflexes cognitifs. Il traite la grande majorité des situations courantes.

- **Système 2 — Lent et Délibéré :** Ce système mobilise une attention et un effort conscient. Il intervient pour les raisonnements complexes, les calculs ou les jugements nuancés qui nécessitent une réflexion approfondie.

Dans KODA, nous avons transposé cette architecture : un agent de classification détermine si la question reçue relève d'un traitement rapide (accès direct à la mémoire) ou d'un traitement approfondi (analyse sémantique complète), et active le pipeline correspondant.

Référence : Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.

Mémoire Épisodique et Mémoire Sémantique

La distinction entre **mémoire épisodique** (souvenirs d'événements vécus) et **mémoire sémantique** (connaissances générales) constitue la base théorique de notre système de mémoire utilisateur. Nous avons organisé la mémoire de KODA selon ces deux catégories : les synthèses journalières constituent la mémoire épisodique, tandis que les centres d'intérêt et points clés forment la mémoire sémantique du profil utilisateur.

Référence : Tulving, E. (1972). *Episodic and semantic memory*. In E. Tulving & W. Donaldson (Eds.), Orgar

Architectures Multi-Agents et Chain-of-Thought

Notre approche s'inscrit dans le domaine des **systèmes multi-agents** (MAS), où des agents spécialisés collaborent pour accomplir une tâche complexe. Chaque agent dans KODA prend en charge une sous-tâche précise (classification, récupération mémorielle, génération de réponse) et transmet son résultat au suivant. Par ailleurs, la technique de **Chain-of-Thought Prompting** — qui consiste à structurer les instructions d'un modèle de langage sous forme de raisonnements explicites étape par étape — améliore significativement la pertinence des réponses générées.

Références : Wooldridge & Jennings (1995), *Intelligent Agents* ; Wei et al. (2022), *Chain-of-Thought Prompt*

Webhook 1 : Analyse et Réponse

Principe Général

Le Webhook 1 est le pipeline principal et en temps réel de KODA. Il est déclenché à chaque fois qu'un utilisateur pose une question au robot. Son rôle est d'analyser la question, d'accéder aux informations contextuelles pertinentes, et de générer une réponse personnalisée et naturelle.

Le pipeline est composé de **6 blocs séquentiels**, chacun accomplissant une tâche précise avant de transmettre ses résultats au bloc suivant. L'ensemble du flux est orchestré par n8n.

Format de la Requête Entrante

Le webhook accepte les requêtes au format `(user_name + question)`. Le champ `user_name` est un identifiant unique généré automatiquement par le backend pour chaque utilisateur en cours d'interaction. Cet identifiant est injecté avec la question dans le flux de travail, permettant d'isoler et de gérer les données de chaque utilisateur de manière indépendante dans les environnements multi-utilisateurs.

Description Détaillée des Blocs de Traitement

Block 1 : Réception et Vérification de l'État du Système

À la réception de chaque requête, la première action du pipeline est de **vérifier si KODA est opérationnel**. Ce bloc contrôle l'état du robot dans la base de données : si KODA est en mode "ouvert" (*open state*), la requête est acceptée et transmise au bloc suivant. Si le système est en pause, en maintenance ou en surcharge, la requête est rejetée et un message approprié est retourné à l'utilisateur.

Ce mécanisme de garde (*gating*) est essentiel pour garantir la stabilité du système et éviter tout traitement dans des conditions non optimales.

Référence : Pattern *Circuit Breaker* — `https://martinfowler.com/bliki/CircuitBreaker.html`

Block 2 : Vérification de la Question dans l'Historique

Avant d'engager le traitement complet de la question, ce bloc effectue une **recherche dans les 5 derniers messages** échangés avec l'utilisateur. L'objectif est de détecter si la question posée est une répétition ou une reformulation d'une question récente déjà traitée.

Le processus fonctionne de la manière suivante :

1. Le système récupère les 5 derniers échanges de l'utilisateur depuis la base de données.
2. Il applique une **comparaison sémantique** entre la nouvelle question et les questions précédentes (vérification de similarité par correspondance de mots-clés ou distance sémantique).
3. **Si une correspondance est trouvée :** la réponse précédente est directement récupérée et renvoyée à l'utilisateur, sans activer le reste du pipeline. Cela évite une consommation inutile de ressources computationnelles et raccourcit drastiquement le temps de réponse.
4. **Si aucune correspondance n'est trouvée :** la question est nouvelle et le pipeline continue normalement vers le Block 3.

Ce bloc d'optimisation est particulièrement utile dans les contextes où plusieurs utilisateurs posent des questions similaires dans un court intervalle, ou lorsqu'un utilisateur répète accidentelle-

ment une question.

Référence : Principe DRY (*Don't Repeat Yourself*) et optimisation de cache — `https://docs.n8n.io/flo`

Block 3 : Classification Sémantique de la Question

Ce bloc est un **agent IA** chargé d'analyser et de classifier la question reçue. Il détermine :

- **La nature de la question :** est-elle factuelle, conversationnelle, émotionnelle, technique ou introductive (présentation de l'utilisateur) ?
- **La complexité du traitement requis :** la question nécessite-t-elle un traitement rapide (Système 1) ou une analyse approfondie (Système 2) ?
- **La nécessité d'accès à la mémoire :** faut-il consulter l'historique et le profil de l'utilisateur pour répondre de manière pertinente ?

En fonction de ces classifications, le nœud de routage dirige la requête vers la branche de traitement la plus adaptée.

Référence : Kahneman, D. (2011). *Thinking, Fast and Slow* ; Russell & Norvig (2010), *Artificial Intelligence*

Block 4 : Reconnaissance et Enregistrement de l'Utilisateur

Si la classification indique que l'utilisateur se présente pour la première fois (type de question : *présentation*), ce bloc est activé. Il **capture l'identité de l'utilisateur** et crée son profil dans la base de données. Cette initialisation est fondamentale : elle prépare la structure de données qui permettra à KODA de mémoriser et d'exploiter les informations de cet utilisateur dans toutes les interactions futures.

Pour les utilisateurs déjà enregistrés, ce bloc est contourné, et la requête est transmise directement au Block 5.

Référence : Tulving, E. (1983). *Elements of Episodic Memory*. Oxford University Press.

Block 5 : Récupération Contextuelle depuis la Base de Données

Lorsque la classification indique la nécessité d'un accès mémoriel, ce bloc interroge la base de données pour collecter l'ensemble des informations pertinentes relatives à l'utilisateur :

- L'historique des échanges récents
- Les centres d'intérêt identifiés
- Les préférences et points clés mémorisés
- Les synthèses journalières générées par le Webhook 2

Ces données constituent le **contexte enrichi** qui sera transmis au générateur de réponse, permet-

tant à KODA de formuler une réponse personnalisée et cohérente avec l'historique de la relation.

Référence : Stickgold, R. (2005). Sleep-dependent memory consolidation. *Nature*, 437, 1272–1278.

Block 6 : Génération de la Réponse par Prompt Engineering

Ce bloc est le nœud de synthèse finale. Le modèle de langage reçoit un **prompt structuré** combinant trois éléments :

1. La question de l'utilisateur
2. Le contexte mémoriel récupéré au Block 5
3. Un ensemble d'instructions précises définissant le rôle de KODA, le ton à adopter et les contraintes de la réponse

Notre stratégie de **Prompt Engineering** s'appuie sur deux techniques complémentaires :

- **Chain-of-Thought :** structurer le raisonnement du modèle étape par étape pour améliorer la cohérence des réponses complexes.
- **Role Prompting :** définir explicitement le rôle et la personnalité de KODA pour garantir un ton naturel et cohérent.

Référence : Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*.

Diagramme de Séquence du Webhook 1

Le diagramme ci-dessous illustre le flux complet des données et des décisions au sein du Webhook 1, depuis la réception de la requête jusqu'à la génération de la réponse finale.

Webhook 2 : Filtre et Synthèse Journalière

Principe et Inspiration

Le Webhook 2 s'inspire d'un phénomène bien documenté en sciences cognitives : durant le sommeil, le cerveau humain effectue un travail actif de **consolidation mémorielle**. Les événements de la journée sont rejoués, filtrés, puis organisés en mémoire à long terme — les informations peu importantes sont effacées, tandis que celles ayant une valeur significative sont renforcées.

Nous reproduisons ce mécanisme de manière algorithmique : le Webhook 2 s'exécute automatiquement chaque nuit (à 02h00), analyse l'ensemble des échanges de la journée pour chaque utilisateur, et en génère une **synthèse structurée** enregistrée dans la base de données.

Référence : Walker, M. & Stickgold, R. (2006). Sleep, Memory, and Plasticity. *Annual Review of Psychology*

Fonctionnement du Webhook 2

Le processus se déroule en trois temps :

1. **Collecte :** Récupération de l'ensemble des nouvelles interactions de la journée pour tous les utilisateurs actifs.
2. **Analyse et synthèse :** Un agent IA analyse le contenu des échanges et génère une fiche de synthèse structurée pour chaque utilisateur.
3. **Persistance :** Les fiches sont enregistrées dans la base de données et associées au profil de chaque utilisateur.

Structure des Fiches de Synthèse Générées

À l'issue de son traitement, le Webhook 2 génère pour chaque utilisateur une fiche de synthèse couvrant **quatre dimensions complémentaires** :

- **Centres d'intérêt :** Identification des sujets qui engagent l'utilisateur, déduits des thématiques abordées dans ses échanges. Cette analyse permet de maintenir un profil d'intérêts à jour.
- **Synthèse de la journée :** Un résumé narratif des échanges significatifs de la journée. Ce résumé constitue la mémoire épisodique condensée de KODA pour cet utilisateur.
- **Points clés :** Extraction des informations factuelles importantes révélées durant la journée — décisions évoquées, projets mentionnés, problèmes soulevés. Ces éléments enrichissent la mémoire sémantique du profil.
- **Conseils personnalisés :** Génération de recommandations adaptées au profil et au contexte de l'utilisateur, permettant à KODA d'offrir une valeur proactive lors des interactions suivantes.

Référence : Miller, G.A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2)

Impact sur le Système Global

Les synthèses du Webhook 2 constituent la **mémoire à long terme** de KODA. Elles sont consultées par le Webhook 1 pour enrichir le contexte des réponses, et exploitées par le Webhook 3 pour générer des interactions spontanées pertinentes.

Webhook 3 : Spontanéité et Interaction Proactive

Principe : La Spontanéité comme Dimension Humaine

Un compagnon humain ne se contente pas de répondre aux questions : il prend aussi l'initiative d'engager la conversation de manière spontanée et contextuelle. Les recherches en psychologie sociale ont montré que les interactions les plus mémorables et satisfaisantes sont souvent celles initiées de façon inattendue, en dehors du cadre purement transactionnel.

C'est cette dimension que le Webhook 3 cherche à reproduire. Son rôle est de permettre à KODA d'**initier lui-même la conversation** lorsqu'il détecte la présence d'un utilisateur qu'il reconnaît, sans attendre que celui-ci pose une question.

Référence : Dautenhahn, K. & Billard, A. (1999). Bringing up robots or the psychology of socially intelligen

Déclenchement et Fonctionnement

Le Webhook 3 est déclenché dès que le **module de vision** de KODA identifie le visage d'un utilisateur reconnu. L'identifiant de cet utilisateur est transmis au webhook, qui active le pipeline de génération d'une phrase d'accroche personnalisée.

Génération de la Phrase d'Accroche

Un agent IA génère une **phrase d'accroche unique et personnalisée** en exploitant les données de la fiche de synthèse produite par le Webhook 2. Les variables injectées dans le prompt de génération sont les suivantes :

Variables du Prompt d'Interaction Spontanée

- `{user_name}` — Prénom ou identifiant de l'utilisateur
- `{interests}` — Centres d'intérêt identifiés (depuis le Webhook 2)
- `{daily_summary}` — Synthèse de la dernière journée d'échanges
- `{key_points}` — Points clés retenus lors des interactions précédentes
- `{personalized_advice}` — Conseils personnalisés disponibles
- `{previous_openings}` — Historique des phrases d'accroche déjà utilisées (pour garantir l'unicité)

Référence : Reeves, B. & Nass, C. (1996). *The Media Equation: How People Treat Computers, Television, a*

Architecture Cognitive Globale de KODA

Les trois webhooks forment ensemble un **système cognitif cohérent**, dont chaque composant reproduit un mécanisme humain identifié et documenté. Le tableau suivant résume ces correspondances :

Table III.1 – Correspondance entre mécanismes cognitifs humains et implémentations de KODA

Mécanisme Humain	Référence Scientifique	Implémentation KODA
Double traitement (S1/S2)	Kahneman, 2011	Block 3 — Classification et routage
Optimisation cognitive	Principe DRY	Block 2 — Vérification historique
Consolidation nocturne	Stickgold, 2005	Webhook 2 — Synthèse journalière
Regroupement mémoriel	Miller, 1956	4 dimensions de la synthèse Webhook 2
Spontanéité sociale	Clark, 1996 ; Dautenhahn & Billard, 1999	Webhook 3 — Interactions proactives

Référence : Russell, S. & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hal

Conclusion

Ce chapitre a présenté l'architecture cognitive de KODA, structurée autour de trois webhooks complémentaires orchestrés par la plateforme n8n. Chaque composant trouve sa justification dans des principes issus des sciences cognitives et de l'intelligence artificielle.

Le **Webhook 1** assure le traitement en temps réel des questions, avec une optimisation par vérification de l'historique (Block 2) et une classification intelligente qui adapte le niveau de traitement à la nature de chaque requête. Le **Webhook 2** reproduit la consolidation nocturne du cerveau humain, en générant chaque nuit des fiches de synthèse personnalisées qui enrichissent progressivement la mémoire à long terme de KODA. Le **Webhook 3** confère au robot une dimension proactive et sociale, lui permettant d'initier des interactions personnalisées dès la détection d'un utilisateur.

Ensemble, ces trois mécanismes — réactif, mémoriel et proactif — forment la fondation de l'expérience utilisateur unique que KODA ambitionne d'offrir : celle d'un compagnon intelligent, attentionné et toujours présent.

CHAPTER IV

SMART GREENHOUSE & IOT INFRASTRUCTURE

Chapter Introduction

This chapter details the physical and logical infrastructure underpinning the Smart Greenhouse layer. It covers the overall IoT system architecture, the hardware components deployed within the greenhouse environment, the communication protocols enabling real-time data exchange, and the practical implementation and wiring of the embedded control system.

IV.1 Overall IoT System Architecture

The IoT architecture follows a strictly decoupled microservices paradigm. The Edge Layer handles physical actuations and raw telemetry via the Raspberry Pi. The Network Layer encapsulates MQTT and WebRTC transports. The Processing Backend operates within Spring Boot, dynamically interacting with the unified Data Layer built on TimescaleDB. This layered design ensures resilience, modularity, and independent scalability of each component.

IV.2 Hardware Components

IV.2.1 Raspberry Pi Integration

The central operational node is a localized Raspberry Pi microcomputer running a headless Linux kernel. Operating completely independently, this unit runs a continuous embedded background daemon utilizing parallel threading. This completely protects sensor polling routines from being artificially stalled by network transmission latency, ensuring consistent actuation regardless of backend availability.

IV.2.2 Sensors and Actuators

The hardware array leverages digital temperature and humidity sensors alongside HC-SR04 ultrasonic mechanisms mapping actual liquid nutritional tank volumes in real-time. Correspondingly, multi-channel 12V relays govern extraction fans, intense photoperiodic lighting mechanisms, and peristaltic pumps dispersing nutrient blends. Each actuator is controlled via GPIO pins mapped through software relay logic running on the Pi daemon.

IV.3 Data Acquisition and Communication

IV.3.1 MQTT Protocol

Message Queuing Telemetry Transport (MQTT) serves explicitly as the IoT transport layer. By prioritizing an asynchronous Publish/Subscribe paradigm across port 8883 (incorporating TLS securing mechanisms), the Pi transacts highly compressed JSON strings representing entire sensor matrices using mere bytes of bandwidth overhead. This lightweight protocol is ideally suited to the constrained network environment of embedded agricultural systems.

IV.3.2 Broker and Data Exchange

An active Mosquitto MQTT Broker independently arbitrates the massive inbound swarm of sensor telemetry, pushing it immediately into the Spring service layers mediating database commits automatically. Topics are structured hierarchically by greenhouse ID, sensor type, and timestamp, enabling efficient fan-out to multiple subscriber services simultaneously. Quality of Service (QoS) levels are configured to guarantee reliable delivery of critical actuation commands.

IV.4 Physical Implementation and Wiring

Physical implementation involved precise GPIO wiring mapping each relay channel to its corresponding actuator. Sensor connections follow strict pull-up resistor configurations to prevent floating pin states. The power distribution layer isolates the 3.3V logic domain of the Pi from the 12V actuator domain via optocoupler-protected relay boards, ensuring full electrical safety and preventing back-EMF damage to the microcontroller. All connections were validated through automated loopback testing scripts prior to field deployment.

Conclusion

This chapter established the complete hardware and communication foundation of the Smart Greenhouse. The Raspberry Pi edge node, combined with the Mosquitto MQTT broker and carefully wired sensor-actuator arrays, forms a robust and autonomous IoT control layer capable of sustaining greenhouse operations entirely independently of cloud connectivity.

CHAPTER V

ARTIFICIAL INTELLIGENCE MODULE (RESEARCH & DEVELOPMENT)

Chapter Introduction

This chapter details the research and development efforts dedicated to the Artificial Intelligence module integrated within the Hydroponic Fodder System. It covers the methodological approach to AI integration, the dataset collection and preprocessing pipeline, the model training and evaluation process, and the deployment strategy enabling real-time inference directly on the edge hardware.

V.1 AI Approach Study

The synergy between robust telemetry (IoT) and cognitive evaluation (AI) enables “predictive” rather than “reactive” farming. Sensors generate raw environmental states, while convolutional AI actively diagnoses early leaf-pathogens directly via camera feeds, ensuring instantaneous mitigating relay actions. The chosen approach prioritizes edge inference over cloud-dependent processing to guarantee sub-second response times and full offline operational capability.

V.2 Data Preparation (Dataset)

V.2.1 Data Collection and Sources

Synthesizing spatial diagnosis algorithms requires massive, balanced datasets. Thousands of pristine and pathogen-infected fodder images were meticulously gathered from both controlled greenhouse environments and publicly available agricultural research databases. Images were captured under varying lighting conditions and growth stages to ensure maximum representational diversity.

V.2.2 Labeling and Preprocessing

All collected images underwent rigorous manual labeling by agricultural domain experts, classifying each sample into healthy or disease-specific categories. Preprocessing steps included standardized resizing to uniform input dimensions, normalization of pixel intensities, and aggressive data augmentation via rotational shifts, horizontal flipping, brightness variance, and contrast adjustments. These augmentation strategies ensured absolute neural robustness when handling chaotic real-world greenhouse lighting conditions.

V.3 Model Development

V.3.1 Training and Testing

The Convolutional Neural Network (CNN) architecture was designed to balance classification accuracy with inference efficiency on constrained embedded hardware. Training was performed across multiple epochs using categorical cross-entropy loss and the Adam optimizer, with early stopping applied to prevent overfitting. The model demonstrated massive operational fidelity, achieving over 94% validation accuracy across all disease categories while retaining sub-three-second physical inference latency entirely onboard the Pi.

V.3.2 Pipeline Edge Impulse

Training operations strictly utilized the **Edge Impulse** framework, aggressively formulated to optimize deep networks explicitly for highly constrained C++ execution on embedded hardware interfaces rather than immense cloud GPUs. The Edge Impulse pipeline automated feature extraction, model quantization, and C++ library generation, producing a deployable inference binary perfectly sized for the Raspberry Pi's memory and processing constraints.

V.4 Edge Deployment and OTA Updates

A crucial engineering hurdle necessitated implementing Over-The-Air (OTA) flashing. As the AI model matures with more data, the Spring Boot backend can securely push binary inference file updates automatically down through the MQTT layer, instantly upgrading the physical field greenhouses globally without requiring destructive manual interventions. Version control mechanisms ensure rollback capability in case a newly deployed model underperforms in field conditions.

Conclusion

This chapter established how the AI module transforms the greenhouse from a reactive system into a predictive diagnostic engine. By combining Edge Impulse-optimized CNN inference with a robust OTA update pipeline, the platform achieves autonomous plant health monitoring that significantly outperforms standard hydroponic systems, positioning it as a genuinely intelligent agricultural solution.

CHAPTER VI

MOBILE APPLICATION (FARMER) & WEB DASHBOARD (ADMIN)

Chapter Introduction

Transforming backend intelligence into accessible and intuitive user experiences characterizes this chapter. It details the implementation of the Flutter mobile application serving the farmer, the React-based web administration dashboard, and the real-time video surveillance system enabling live greenhouse monitoring from any remote location.

VI.1 Mobile Application Interface

Authored within **Flutter / Dart**, the mobile gateway boasts a dynamic "Glassmorphism" UI aesthetic. The application operates strictly as the farmer's primary operational tool, utilizing asynchronous state management to paint real-time telemetry changes natively on the screen instantly, while simultaneously surfacing complex AI diagnostic notifications generated from the backend.

VI.1.1 Real-Time Monitoring

The monitoring screen renders live sensor readings through animated gauge widgets and historical trend charts. Data is pushed directly via WebSocket connections established with the Spring Boot backend, ensuring that all displayed values reflect the current physical state of the greenhouse with minimal latency. Color-coded threshold indicators instantly signal abnormal conditions to the farmer.

VI.1.2 Notifications and Alerts

The notification engine delivers push alerts via Firebase Cloud Messaging (FCM), ensuring farmers receive critical disease detection warnings even when the application is running in the background. Alert payloads include the detected disease type, confidence score, affected greenhouse zone, and recommended corrective actions, empowering immediate informed responses.

VI.2 Web Administration Dashboard

Operating across React.js structures, the web portal acts autonomously catering to the host enterprise administrators. This view implements complex data grids enabling massive multi-tenant views, mapping global greenhouse uptimes flawlessly and allowing deep historic reporting exportations safely.

VI.2.1 User and Device Management

The administration panel provides full CRUD capabilities for user account management, greenhouse assignment, and device configuration. Role-based access control ensures strict separation between farmer and administrative permissions, preventing unauthorized access to system-critical configuration parameters.

VI.2.2 Data Visualization (Grafana)

Integrating **Grafana** securely onto the TimescaleDB outputs guarantees breathtaking, deeply interactable charting interfaces. Grafana rapidly compiles historic temperature aggregations, humidity trends, and nutrient level histories, delivering instant analytical curves highlighting long-term system behavioral drifts seamlessly to authorized super-users.

VI.3 Video Surveillance System

VI.3.1 Architecture WebRTC / Coturn

To provide flawless visual streaming from the greenhouse camera directly to the farmer without experiencing brutal transcoding latencies, the architecture implements direct **WebRTC** streaming. Facilitated fundamentally using a **Coturn** TURN/STUN relay server configuration, the video perfectly navigates complex firewall NAT boundaries establishing hyper-secure peer-to-peer data links generating zero-lag visual supervision globally.

VI.3.2 Live Greenhouse Monitoring

The live monitoring view is embedded directly within both the mobile application and the web dashboard, presenting a real-time MJPEG or WebRTC video feed from the Raspberry Pi camera. Resolution and bitrate are dynamically adjusted based on available network bandwidth, ensuring a stable viewing experience across both high-speed Wi-Fi and constrained mobile data connections.

Conclusion

The implementation phase validated the architectural coherence of the platform, successfully unifying WebRTC media streaming protocols within cross-platform Flutter applications. The combination of real-time telemetry, AI-driven alerts, live video, and comprehensive administration tools delivers a complete and professional user experience for both farmer and administrator roles.

CHAPTER VII

DEPLOYMENT AND CLOUD INTEGRATION

Chapter Introduction

This final chapter addresses the cloud deployment strategy, backend infrastructure, data management, security architecture, and comprehensive system validation. It demonstrates how all individual layers of the platform converge into a unified, production-grade system ready for real-world agricultural deployment.

VII.1 Cloud Deployment Architecture

The production deployment leverages a containerized microservices architecture orchestrated via Docker and Docker Compose. Each system component (Spring Boot backend, Mosquitto broker, TimescaleDB, Grafana, Coturn) runs within isolated containers, enabling independent scaling, simplified maintenance, and rapid redeployment across any Linux-compatible cloud virtual machine. The deployment target is a dedicated VPS instance providing the static IP required for STUN/TURN server operation.

VII.2 Backend & Database

VII.2.1 Architecture Spring Boot

The server environment relies exclusively upon the **Spring Boot** Java framework. This provides unparalleled enterprise-grade dependency injection, absolute concurrency thread management, and natively secures vast API arrays utilizing standard Spring Security JWT filters handling authorization logic flawlessly. RESTful API endpoints are versioned and documented via Swagger/OpenAPI, facilitating seamless integration with both the mobile and web clients.

VII.2.2 Time-Series Data Management (TimescaleDB/PostgreSQL)

Traditional relational models struggle when ingesting millions of telemetry data points daily. Consequently, the implementation relies on **TimescaleDB**, an extension of PostgreSQL that maintains full ACID compliance for relational data (user accounts, device IDs) while converting telemetry tables into hyper-tables. These hyper-tables automatically partition environmental sensor data across temporal chunks, significantly accelerating historical queries and enabling instant generation of statistical graphs tracking humidity variations over entire crop seasons without exhausting backend memory.

VII.3 Security and Authentication

Security is enforced across all communication layers. JWT-based authentication secures all REST API endpoints, with token expiration and refresh mechanisms preventing session hijacking. All IoT communications transit over MQTTS (port 8883) with mutual TLS certificate validation, preventing unauthorized device connections. Video streams are protected via WebRTC DTLS-SRTP encryption, ensuring that live greenhouse footage remains inaccessible to unauthorized parties. Additionally, all user passwords are stored as bcrypt hashes with configurable salt rounds, complying with modern security standards.

VII.4 Full System Validation and Testing

Comprehensive JUnit tests were developed to rigorously validate backend API controllers. Simulated heavy-load injection scripts executed thousands of concurrent JWT authentication requests, verifying that Spring Boot security filters correctly blocked unauthorized access. Hardware testing involved deliberately disconnecting network connectivity to evaluate local fallback mechanisms and confirm autonomous offline actuation. Simultaneously, the Edge Impulse CNN demonstrated reliable real-world validation, accurately distinguishing fungal variations under volatile natural lighting conditions when deployed on the edge Pi. All non-functional requirements defined in Chapter II were verified and confirmed within their target thresholds.

Conclusion

This chapter demonstrated that the Hydroponic Fodder System successfully meets all functional and non-functional requirements through rigorous cloud deployment, enterprise-grade backend engineering, and comprehensive security and testing protocols. The platform is validated, containerized, and production-ready for real-world agricultural deployment.

GENERAL CONCLUSION AND FUTURE PERSPECTIVES

This project addressed critical agricultural challenges by engineering the complete **Hydroponic Fodder System**. Developed under MicroEdition's strategic guidance, the project successfully integrated edge IoT operations with robust Spring Boot cloud services, bridging TimescaleDB time-series data with intuitive Flutter interfaces to achieve truly autonomous greenhouse capabilities independent of manual labor.

The project produced significant technological outcomes, aggregating thousands of sensor metrics via MQTT streams. The Edge Impulse AI model surpassed a 94% recognition threshold, autonomously detecting fungal patterns faster than human perception. Integrating Coturn/WebRTC delivered near-zero-latency streaming capabilities, enhancing user oversight through remote verification of physical component actuations.

Summary of Achievements:

- Full-stack IoT greenhouse control system deployed on Raspberry Pi with MQTT communication.
- CNN-based plant disease detection model achieving over 94% accuracy with edge inference.
- Flutter mobile application with real-time telemetry monitoring and AI push alerts.
- React web administration dashboard with Grafana data visualization.
- Live WebRTC video surveillance with Coturn TURN/STUN relay.
- Containerized cloud deployment with Spring Boot, TimescaleDB, and JWT security.
- OTA model update pipeline enabling continuous AI improvement without manual intervention.

Limitations: Despite formidable achievements, the current hardware footprint remains bound by local power grid dependencies, defining slight limitations for fully off-grid deployment. The AI model's accuracy is also constrained by the size and diversity of the training dataset, which may not fully generalize to all fodder species or disease variants.

Future Perspectives: Future development will prioritize integrating solar photovoltaic systems with lithium battery storage, enabling fully off-grid operation across remote agricultural sites. Additionally, expanding the AI capabilities to support autonomous pesticide injection based on disease recognition would further enhance operational efficiency. Scaling the multi-tenant platform to support federated greenhouse networks, enriched by continuous federated learning pipelines that improve the shared disease detection model, represents a significant step toward revolutionizing automated intelligent crop production globally.

BIBLIOGRAPHY

[B1] SPERRY, R. W., “Hemisphere deconnection and unity in conscious awareness”, *American Psychologist*, 23(10), 723–733, 1968.

[B2] GAZZANIGA, M. S., “Cerebral specialization and interhemispheric communication: Does the corpus callosum enable the human condition?”, *Brain*, 123(7), 1293–1326, 2000.

[B3] GAZZANIGA, M. S., *The Social Brain: Discovering the Networks of the Mind*, Basic Books, 1985.

[B4] KAHNEMAN, D., *Thinking, Fast and Slow*, Farrar, Straus and Giroux, 2011.

[B5] STANOVICH, K. E., & WEST, R. F., “Individual differences in reasoning: Implications for the rationality debate?”, *Behavioral and Brain Sciences*, 23(5), 645–665, 2000.

[B6] TULVING, E., “Episodic and semantic memory”, in E. Tulving & W. Donaldson (Eds.), *Organization of Memory* (pp. 381–403), Academic Press, 1972.

[B7] TULVING, E., *Elements of Episodic Memory*, Oxford University Press, 1983.

[B8] STICKGOLD, R., “Sleep-dependent memory consolidation”, *Nature*, 437(7063), 1272–1278, 2005.

[B9] WALKER, M. P., & STICKGOLD, R., “Sleep, memory, and plasticity”, *Annual Review of Psychology*, 57, 139–166, 2006.

[B10] MILLER, G. A., “The magical number seven, plus or minus two: Some limits on our capacity for processing information”, *Psychological Review*, 63(2), 81–97, 1956.

[B11] RUSSELL, S., & NORVIG, P., *Artificial Intelligence: A Modern Approach (3rd ed.)*, Prentice Hall, 2010.

[B12] WOOLDRIDGE, M., & JENNINGS, N. R., “Intelligent agents: Theory and practice”, *The Knowledge Engineering Review*, 10(2), 115–152, 1995.

[B13] BROWN, T. B., et al., “Language models are few-shot learners”, *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 1877–1901, 2020.

[B14] WEI, J., et al., “Chain-of-thought prompting elicits reasoning in large language models”, *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 2022.

[B15] CLARK, H. H., *Using Language*, Cambridge University Press, 1996.

[B16] DAUTENHAHN, K., & BILLARD, A., “Bringing up robots or the psychology of socially intelligent robots”, *Proceedings of the Third Annual Conference on Autonomous Agents* (pp. 366–367), ACM, 1999.

[B17] REEVES, B., & NASS, C., *The Media Equation: How People Treat Computers, Television, and New Media Like Real People and Places*, Cambridge University Press, 1996.

WEBOGRAPHY

[W1] WEBRTC.org, “Real-Time Communication APIs”, in *WebRTC Architecture Docs*, `https://webrtc.org`, accessed March 2026.

[W2] FLUTTER NATIVE TEAM, “Flutter – Reactive Component Layout Mechanisms”, in *Flutter.dev*, `https://flutter.dev/docs`, accessed March 2026.

[W3] ECLIPSE FOUNDATION, “Mosquitto MQTT Broker API parameters”, in *Eclipse Mosquitto*, `https://mosquitto.org/documentation/`, accessed February 2026.

[W4] GRAFANA LABS, “Grafana Visualizing Time-Series Data”, in *Grafana Documentation*, `https://grafana.com`, accessed February 2026.